ASAI Taj Delhi — Work Log & Replication Guide

Date: Tuesday, 21 July 2026 · Updated: ~03:40 IST · Project: **asai-taj-delhi** · Account: melvai@gmail.com · Repo: `sketch_jun19a`

Purpose: Single record of today's Firebase + live dashboard setup for replication and Claude archive.

> **Current status:** Firmware `asai_dashboard` **v4.0** publishes WiFi-direct to Taj RTDB (`/ASAITajDelhi/*`). USB bridge `serial_to_taj.py` is fallback only — stop it when WiFi firmware is live (avoid double writes). Social untouched.

1. Outcome (what exists now)

- **Firebase project** `asai-taj-delhi` (Spark plan, Analytics off)
- **Realtime Database** region `asia-southeast1` (Singapore) — Mumbai `asia-south1` is *not* offered for RTDB
- **Hosting live:** https://asai-taj-delhi.web.app
- **Dashboard focus:** lighting (AS7341 spectral), sound, health scores, compliance, CPCB AQI, Delhi outdoor vs Taj indoor (US AQI)
- **Live data path (preferred):** ESP32 WiFi → Firebase_ESP_Client → `/ASAITajDelhi/live` + `/log`
- **Fallback:** USB Serial → `scripts/serial_to_taj.py` → same paths
- **Removed:** hotel website folders (`website/`, `website-heritage/`, `website-comforts-backup/`)

2. Firebase project setup (console)

1. Go to `console.firebase.google.com` → Add project → name `asai-taj-delhi`
2. Skip Google Analytics
3. Build → Realtime Database → Create → region **asia-southeast1** → start in **test mode**
4. Build → Hosting → Get started (or init via CLI)
5. Project settings → Add app → Web → nickname e.g. `asai-taj-delhi-web` → copy `firebaseConfig`

firebaseConfig (web)

```
const firebaseConfig = {
  apiKey: "AIzaSyC5rCvD8LNKzgr6d3ZA1Gl-pLfi4gAn0x8",
  authDomain: "asai-taj-delhi.firebaseapp.com",
  databaseURL: "https://asai-taj-delhi-default-rtdb.asia-southeast1.firebasedatabase.app",
  projectId: "asai-taj-delhi",
  storageBucket: "asai-taj-delhi.firebasestorage.app",
  messagingSenderId: "459317113967",
  appId: "1:459317113967:web:10015c5316759dd5af899a"
};
```

> **Security:** RTDB was opened in test mode (time-limited open read/write). Lock rules before any public/client-facing launch.

3. Local repo / Hosting config

File	Role
`.firebaserc`	Default project `asai-taj-delhi`
`firebase.json`	Hosting public folder = `public/`
`public/index.html`	Full live dashboard (single file)
`asai_dashboard/asai_dashboard.ino`	ESP32 firmware v4.0 — sensors + OLED + WiFi → Taj RTDB
`asai_dashboard/secrets.h`	Local WiFi SSID/password (gitignored); copy from `secrets.h.example`
`scripts/serial_to_taj.py`	USB serial → Taj *fallback*; stop when WiFi firmware is publishing
`scripts/simulate-live.js`	Dev simulator only — **stopped**; do not run alongside real ESP32

CLI commands to replicate deploy

```
cd /path/to/sketch_jun19a
firebase login          # if needed
firebase use asai-taj-delhi
firebase deploy --only hosting
```

4. Live data — WiFi-direct firmware (preferred)

Do not use Social. Social Proto2 is a separate project. Taj ESP32 publishes with Mobizt `Firebase_ESP_Client` (same library pattern as Social) but only to `asai-taj-delhi` / `/ASAITajDelhi/*`.

Library

Arduino Library Manager: **Firebase Arduino Client Library for ESP8266 and ESP32** (Mobizt).

Flash steps

1. Edit `asai_dashboard/secrets.h` — set `WIFI_SSID` / `WIFI_PASSWORD` (from `secrets.h.example`)
2. Flash `asai_dashboard.ino` (ESP32-S3)
3. Serial @ 115200: look for `WiFi OK`, `Firebase → asai-taj-delhi ready`, then `cloud: Taj OK`
4. **Stop** `serial_to_taj.py` if running — one writer only
5. Open asai-taj-delhi.web.app — payload includes `source: "esp32_wifi"`

```
# Paths written by firmware every ~5s
setJSON  /ASAITajDelhi/live
pushJSON /ASAITajDelhi/log
# Toggle offline sensors-only: #define ENABLE_CLOUD 0
```

Fallback — USB serial bridge

1. Flash / run firmware (sensors → Serial @ 115200)
2. **Close Arduino Serial Monitor** (port free — typically `/dev/cu.usbmodem2101`)

3. Run the bridge:

```
python3 scripts/serial_to_taj.py
# PUT  /ASAITajDelhi/live.json
# POST /ASAITajDelhi/log.json
# Do not run alongside WiFi firmware
```

Serial line formats the bridge parses

```
Lux 185
415 violet |####...|  420
...
nir/clear 0.12
CCT ~3200K (calibrated)  ->  warm LED
IAQ 32 (acc 2) Excellent | CO2eq 620ppm | VOC 0.35ppm | 23.4C 48%
PM2.5 28 (SI 47) | PM10 52 (SI 51) -> AQI 51 [Satisfactory] driven by PM10
Mic -45.2 dBFS (~60 dBA)
```

> **Port busy?** If you see `Resource busy`, quit Serial Monitor (or any other process on the USB modem), then restart `serial_to_taj.py`.

5. RTDB data paths

Path	Write pattern	Purpose
`/ASAITajDelhi/live`	overwrite (~5s)	Dashboard `.on('value')`
`/ASAITajDelhi/log`	pushJSON	Timeline history
`/ASAITajDelhi/delhi_outdoor`	overwrite	Connaught Place outdoor US AQI (aqi.in ground snapshot)

Live payload fields

Field	Source	Notes
`temperature`, `humidity`	BSEC	From IAQ serial line
`iaq`, `iaq_accuracy`, `eCO2`, `voc`	BSEC	
`noise_level`	INMP441	approx dBA
`light_intensity`	BH1750	lux
`cct`, `light_type`, `nir_ratio`	AS7341	
`f1…f8`, `bands[]`	AS7341	Spectral
`pm25`, `pm10`, `aqi`, `aqi_category`, `aqi_dominant`, `si25`, `si10`	PMS7003	CPCB
`timestamp`	ESP32 NTP (or bridge)	epoch ms

`source`	firmware	`esp32_wifi` when on-device publish

6. Dashboard sections

1. **Health score** — comfort / wellness / acoustics / lighting
2. **Compliance grade** A–F — zone: Lobby, Dining, Bar, Corridor, Meeting
3. **Live telemetry** — temp, RH, lux, CCT, sound, CPCB AQI, BSEC IAQ, eCO_2
4. **AS7341 spectral color** — CCT, NIR/Clear, F1–F8 bars
5. **CPCB AQI panel** — index, PM2.5/PM10, sub-indices
6. **Delhi outdoor vs Taj indoor** — *US AQI* (same scale as aqi.in)
7. **Lighting suitability** + **acoustic zones**
8. **Timeline** + zone compliance checklist

7. Critical lesson: AQI 51 vs aqi.in ~140

Root cause: First outdoor comparison used Open-Meteo modelled PM with *CPCB* math → ~51. aqi.in Connaught Place defaults to *US AQI* (~144) with ground PM2.5 ~53 μg/m³.

- Comparison uses **US AQI** (EPA), labeled to match aqi.in
- Outdoor prefers RTDB `/ASAITajDelhi/delhi_outdoor` when fresh (seeded from aqi.in: us_aqi 144, pm25 53, pm10 88)
- Fallback: Open-Meteo `us_aqi` field (not CPCB-from-model-PM)
- Taj indoor CPCB stays on the AQI panel; comparison converts PMS PM2.5 → US AQI
- Reference: [aqi.in Connaught Place]

8. Template lineage

- **UI / scoring shell:** Social Thane dashboard (`~/Desktop/AetherSensAI/Social/index.html`) — used as UI pattern only
- **Compliance letter idea:** Phygital `audit.html`
- **Sensor suite:** `asai_dashboard.ino` (BH1750, AS7341, BME/BSEC, INMP441, PMS7003)
- **Explicit constraint:** Do not write to or retarget Social Firebase (`asaisocialproto2`)

9. Timeline of data sources today

Phase	Source	Notes
1	One-shot RTDB seed	Static snapshot — telemetry did not move
2	`simulate-live.js`	Fake drifting values every 5s — stopped
3	`serial_to_taj.py` + ESP32	USB bridge — real sensors → Taj

4	`asai_dashboard` v4.0 WiFi	**Preferred** — ESP32 publishes Taj RTDB directly

10. Next steps

1. Edit `secrets.h`, flash v4.0, confirm `cloud: Taj OK` on Serial
2. Stop USB bridge when WiFi publish is confirmed
3. Refresh `delhi_outdoor` periodically from aqi.in / trusted ground API
4. Lock RTDB security rules before guest-facing use

11. Quick reference URLs

What	URL
Live dashboard	https://asai-taj-delhi.web.app
Firebase console	https://console.firebase.google.com/project/asai-taj-delhi/overview
RTDB	https://asai-taj-delhi-default-rtdb.asia-southeast1.firebasedatabase.app
Delhi outdoor reference	https://www.aqi.in/in/dashboard/india/delhi/new-delhi/connaught-place

---

Generated 21 Jul 2026 · updated with WiFi-direct Taj publish (v4.0) · Melroy D’Mello · ASAI Taj Delhi · Share this PDF into Claude Projects / memory as the replication record.